

Mi Tutora Website deep codebase audit

Audit date: 2026-08-09 Scope: the complete committed repository excluding generated documentation as an implementation source Method: repository-wide static analysis, end-to-end flow tracing, mutation-order review, dependency/asset inspection, and a separate adversarial second pass Constraint: application code was not modified

Executive summary

Metric	Count
Total issues	58
Critical	1
High	15
Medium	25
Low	13
Informational	4

The dominant risk is that identity-adjacent, marketplace, payment, wallet, withdrawal, and lifecycle operations execute directly in untrusted browser code. The production Firestore Rules are not present, so the repository cannot demonstrate a secure authorization boundary. Independently of rules, the current payment experience does not integrate a payment processor: it writes “paid” flags and wallet balances directly.

The most immediate data-integrity defects are in grouping. The group UI returns `groupId` while its persistence caller reads `groupDocId`; the synchronization helper reads `studentIds / groupId` while current creation writes `studentDocIds / groupDocId`; and new students are linked to a temporary group ID while the group is saved under a different Firestore auto-ID. These are executable mismatches capable of clearing learner links, deleting valid groups, and orphaning tuition requests.

Authentication initialization is split across email signup, Google signup, login, dashboard fallback creation, and silent form replay. Those paths disagree about `student` versus `parent`, UID versus generated profile document IDs, and singular versus plural roles. Multi-document business operations use sequential writes without transactions or idempotency, so partial failures and conflicting clients can persist impossible states.

Issue index

ID	Severi ty	Area	Issue	Impact	Evidence	Recommendation
AU D- 001	Critical	Payme nts	Payment success, wallet use, earnings and withdrawal are unverified client writes	Financial manipulation/loss and false payment state	student dashboard:1024-1127; teacher dashboard:302-369,670-693	Move all money state behind verified provider/server workflows and an immutable ledger

ID	Severity	Area	Issue	Impact	Evidence	Recommendation
AU D-002	High	Authorization	Users can append their own selected role; missing user docs gain the dashboard role	Privilege escalation if rules allow these writes	login:60-75,250-281; dashboardApi:16-34,245-264	Make role grants server/admin controlled
AU D-003	High	Firebase security	No Firestore/Storage rules are versioned while clients perform privileged writes	Effective production authorization cannot be proven or reviewed	repository inventory; firebase client:1-18	Version, test and deploy least-privilege rules; use trusted server mutations
AU D-004	High	Groups	GroupManager emits <code>groupId</code> , caller reads/writes <code>groupDocId</code>	Saving groups can clear every learner group assignment and delete groups	GroupManager:188-200; student dashboard:1991-2009,200-255	Use one typed canonical field and transactional migration
AU D-005	High	Groups	Synchronizer reads legacy fields and treats current groups as empty	A valid current-format group can be deleted	groupUtils:12-24,48-60; DemoForm:640-654	Support canonical <code>studentDocIds</code> / <code>groupDocId</code> ; never infer deletion from alias absence
AU D-006	High	Onboarding	New students retain temporary group IDs while groups use unrelated auto-IDs	Student, group and request records do not join	DemoForm:577-618,638-676	Allocate the canonical group document ID first and write it everywhere atomically
AU D-007	High	Identity model	Auth paths create parent/tutor documents under UID or generated IDs inconsistently	Dashboard queries, handlers, listeners and deletion miss profiles/records	login:266-278; signup:72-79,125-173; dashboardApi:37-38,267-272	Canonicalize role-profile IDs and migrate legacy documents
AU D-008	High	Data integrity	Application creation and lifecycle changes are non-atomic multi-document writes	Orphan applications and stale pending queues after partial failure	student dashboard:704-737; teacher dashboard:449-484; decline handlers	Execute one authorized transactional/idempotent workflow
AU D-009	High	Withdrawals	Withdrawal creation and wallet zeroing are separate and retryable	Duplicate payouts, lost balance or spend-after-request races	student dashboard:1111-1127; teacher dashboard:677-693	Reserve funds and create a unique payout request transactionally on server
AU D-010	High	Account deletion	Data is deleted before Auth; recent-login failure is reported as success	Orphan Auth account, partial erasure and misleading compliance outcome	student dashboard:3577-3631; teacher dashboard:2948-2992	Reauthenticate first and orchestrate auditable server-side deletion
AU D-011	High	Signup reliability	Auth account is created before Firestore/profile/verification steps with no rollback/resume	Stranded "email already in use" accounts and incomplete profiles	signup:28-97	Implement idempotent resumable onboarding and cleanup/recovery
AU D-012	High	Form replay	Stored form replay is non-idempotent and only clears storage after all writes	Effect replay/refresh after partial failure duplicates learners and requests	student dashboard:474-567; teacher dashboard:218-272	Add submission IDs, transaction/status records and cancellation guards
AU D-013	High	Hiring	Competing application cleanup queries legacy <code>groupId</code> / <code>studentId</code>	Other tutors remain active after one tutor is hired	student dashboard:980-1014,1057-1091 vs application writes:711-712,842-843	Query canonical participant/group fields in a server transaction

ID	Severity	Area	Issue	Impact	Evidence	Recommendation
AU D-014	 High	Learner deletion	Removal deletes applications by legacy <code>studentId</code> and does not clean pending queues	Orphan applications and sensitive stale references	student dashboard:3458-3481	Resolve all canonical references and clean both sides atomically
AU D-015	 High	Concurrency	Lifecycle transitions are unrestricted last-write-wins client updates	Conflicting accept/decline/pay actions can overwrite each other	student dashboard:881-958; teacher dashboard:603-664	Enforce versioned state transitions with transactions/preconditions
AU D-016	 Medium	Roles	Adding a second role can create <code>roles</code> containing only the new role	Original role becomes unauthorized on a later fetch	login:63-76; signup:114-123	Backfill both legacy role and new role in one canonical role array
AU D-017	 Medium	Signup	Normal consumer role is <code>student</code> , but profile creation checks <code>parent</code>	Email/Google signup omits expected parent stub	signup:25,72-79,125-131,168-174	Align product vocabulary and initialization logic
AU D-018	 Medium	Silent onboarding	Silent student replay creates per-student requests, no group documents, and uses top-level category	Data shape differs from DemoForm's normal dashboard submission	student dashboard:497-556	Route replay through the same canonical onboarding service
AU D-019	 Medium	Profile editing	DemoForm reads <code>parentId</code> , <code>studentId</code> , <code>groupId</code> aliases not used by current writes	Existing data/preferences fail to load or update	DemoForm:143-165,171,526,550	Query canonical links with explicit legacy fallback/migration
AU D-020	 Medium	Realtime	Teacher listener filters by UID while fetcher may resolve generated tutor document ID	Applications can load initially but never refresh in realtime	dashboardApi:267-272; teacher dashboard:274-286	Listen using the resolved canonical tutor document ID
AU D-021	 Medium	Async cleanup	Dynamic-import listener setup can finish after effect cleanup	An unmounted dashboard may retain a Firestore listener	student dashboard:569-585; teacher dashboard:274-290	Add cancellation flag before attaching; immediately unsubscribe if cancelled
AU D-022	 Medium	Realtime errors	Snapshot listeners omit explicit error handling/user recovery	Permission/network listener failure silently stops live refresh	student dashboard:576-579; teacher dashboard:281-284	Supply error callback, telemetry and visible retry state
AU D-023	 Medium	Expiry	Dashboard fetchers perform time-based writes using client clocks	Reads require write permission; clock skew and inactivity change business results	dashboardApi:112-165,444-498	Move expiry to scheduled trusted backend logic
AU D-024	 Medium	Demo timing	"Demo end" is actually parsed start time and parsing trusts loose strings	Decision window begins too early; invalid time/date yields inconsistent expiry	dashboardApi:137-165,470-498	Store timestamps/end duration and validate server-side
AU D-025	 Medium	Expiry concurrency	Both role fetchers can concurrently run the same expiry/queue cleanup	Duplicate writes, partial cleanup and extra costs	dashboardApi duplicated expiry blocks	Make expiry idempotent and single-owner with transactional preconditions
AU D-026	 Medium	Limits	Daily, queue and demo limits use stale cached client data	Parallel tabs/devices can exceed business limits	student dashboard:606-668,748-818; teacher dashboard:374-424,495-547	Enforce limits transactionally at write boundary

ID	Severity	Area	Issue	Impact	Evidence	Recommendation
AU D-027	Medium	Duplicate actions	React loading state is not an idempotency lock and some actions lack loading guards	Rapid clicks/retries can create duplicate applications or transitions	offer/request handlers and hire confirmation:3525-3533	Disable synchronously with request token and server idempotency key
AU D-028	Medium	Student payment	Wallet is deducted before application/payment updates	Later failure reduces balance without completing intended action	student dashboard:1030-1053	Commit ledger and application transition atomically after verification
AU D-029	Medium	Payment history	Writer appends <code>subsequentPayments</code> ; Uls read <code>paymentHistory</code>	Received payments never appear in displayed history	teacher dashboard:351-363,2463,2658-2675; student dashboard:3288-3318	Define one typed payment-history model
AU D-030	Medium	Dues	Projected upcoming-class objects omit <code>feePaid</code> and <code>startDate</code> , but dues reads them	Overdue detection/payment prompts are incorrect or never trigger	dashboardApi:218-231; student dashboard:158-166,2561-2565	Preserve fields or read <code>cls.app</code> through a typed projection
AU D-031	Medium	Earnings	Earnings count tuition-started as revenue regardless of verified fee and estimate demo fees/dates	Misleading financial reporting	dashboardApi:522-588	Derive reports only from verified ledger transactions
AU D-032	Medium	Negotiation	Tutor direct-demo bounds are 60%-120%; priced-offer bounds are 100%-140%	Same parties receive inconsistent allowed prices by entry path	teacher dashboard:425-464 vs 555-573	Centralize and product-approve one ruleset
AU D-033	Medium	Matching	Fetch filtering and scoring use different OR/AND and normalization rules	"Recommended" results are inconsistent and hard to explain	dashboardApi:67-96,347-418; matching.ts:30-95	Build one tested matcher and expose match reasons
AU D-034	Medium	Applications	Teacher-created group apps put group ID in <code>studentDocId</code>	Primary student enrichment fails and downstream fallback behavior varies	teacher dashboard:456-459,564-567; dashboardApi:442-505	Keep group and student identifiers semantically distinct
AU D-035	Medium	Notifications	Notification documents are written but never read by website	Dead persistence, no durable delivery/read behavior	student dashboard:1000-1010,1077-1087; dashboardApi notification derivation	Define one notification source and consumer contract
AU D-036	Medium	Privacy/scaling	Teacher dashboard reads every student and global hired/locked applications	Read costs and exposure grow with whole database	dashboardApi:275-314	Publish sanitized paginated leads and scoped lock queries
AU D-037	Medium	Network performance	Every application snapshot triggers a full multi-query dashboard refetch	Write bursts cause read amplification and UI churn	dashboard listeners; dashboardApi full aggregations	Patch cache from snapshot or invalidate narrower paginated queries
AU D-038	Medium	Pagination	Tutor/student marketplaces and applications are unbounded and rendered as full lists	Increasing latency, memory, Firestore cost and DOM work	dashboardApi:56-59,314-319; dashboard list maps	Add cursor pagination and result limits
AU D-039	Medium	Redirects	<code>next</code> query value is sent directly to router navigation	Open redirect/phishing and unexpected navigation surface	login:83-85,284; signup:182-184	Allow only normalized same-origin route paths

ID	Severity	Area	Issue	Impact	Evidence	Recommendation
AU D-040	Medium	URLs/modal state	Meeting links accept any parseable scheme and link/platform state is not reset	Unsafe/wrong stale meeting destination can be stored and opened	ActionModal:39-49,69-80; dashboard links	Allowlist HTTPS/known hosts and reset all modal state
AU D-041	Low	Environment	Firebase config has no presence/shape validation	Missing values fail late with opaque Firebase errors	firebase client:5-18	Validate required public configuration at startup/build
AU D-042	Low	Geocoding	Nominatim requests lack <code>res.ok</code> , schema checks, cancellation and fetch timeout	Hangs, malformed responses and post-unmount updates degrade forms	TeacherForm:138-170; DemoForm:250-286; GroupSettings:43-76	Wrap in cancellable typed client with timeout/error handling
AU D-043	Low	Referrals	Referral codes are non-unique and first match wins; invalid codes are silently retained	Wrong attribution or dead referral metadata	referral.ts; signup:47-69,143-165	Reserve unique codes and explicitly validate/reject invalid input
AU D-044	Low	Referrals	Referral generation never resets its in-progress flag on failure	No automatic retry during that mount	student dashboard:448-472; teacher dashboard:187-210	Reset in <code>finally</code> and expose retry
AU D-045	Low	Error UI	Student dashboard has no equivalent non-auth fetch error screen	Persistent load/empty UI gives weak recovery feedback	student dashboard:145-153,1149-1153 vs teacher:667-668	Add route error state and retry action
AU D-046	Low	Assets	<code>/book.png</code> is 9.68 MB and rendered through raw <code></code>	Slow marketplace load and excess bandwidth	public/book.png; dashboard image usages	Compress/resize and use Next Image or optimized variants
AU D-047	Low	Dependencies	Many large dependencies are unused; Axios imports are unused	Larger install/audit surface and lint noise	package.json; login:6; both dashboards:7	Remove only after usage/bundle verification
AU D-048	Low	SEO	Sitemap contains only home and login despite other public routes	Signup/legal discovery and freshness metadata are incomplete	sitemap.ts:3-17	Include intended indexable routes and stable last-modified values
AU D-049	Low	Product consistency	FAQ says demo is completely free while tutor UI requires demo payment	User/support dispute and unclear fee ownership	FAQ:40-42; teacher dashboard:302-340	Reconcile copy, policy and implemented charging model
AU D-050	Low	Encoding	Source contains mojibake for currency, copyright and emoji text	Broken production labels and unprofessional UI	ActionModal:137; login copyright; multiple source comments/text	Normalize repository encoding to UTF-8 and add visual checks
AU D-051	Low	Analyzer	Analyzer expects a fixed credential filename and catches failure without nonzero exit	Credential-handling risk and false-success automation	db_analyzer/analyze.js:5-18,112-130	Use ADC/env path, explicit ignore and nonzero exit
AU D-052	High	Testing	No app, rule, integration or E2E tests cover critical security/money/data flows	Regressions in already divergent duplicated logic are likely undetected	package.json scripts; repository inventory	Add emulator rule tests and risk-first integration/E2E suite

ID	Severity	Area	Issue	Impact	Evidence	Recommendation
AU D-053	Low	Toolchain	Node/package-manager versions and CI are not pinned/configured	Non-reproducible local and deployment behavior	no engines/version/CI file	Pin supported runtime and add CI lint/build/test gates
AU D-054	Low	Browser storage	Logout clears all local/session storage for the origin	Discards unrelated form/category preferences and future cohosted state	student dashboard:588-594; teacher:293-299	Remove only application-owned keys
AU D-055	Informational	Architecture	No application backend/API routes/Functions/Storage/Messaging implementation exists	All trusted behavior must be external or is absent	repository inventory	Preserve this fact in design decisions
AU D-056	Informational	Build verification	Dependency installation timed out; lint/build binaries remain unavailable	Compile/lint status could not be established in this environment	attempted <code>npm install</code> , <code>npm run lint</code> , <code>npm run build</code>	Run clean install and checks in functioning CI/network environment
AU D-057	Informational	Schema evidence	Schema report is sampled, root-only and not a contract	Rare fields/subcollections may be absent from audit	db_analyzer:67-108; schema report header	Validate against rules/emulator/export before migration
AU D-058	Informational	Production unknowns	Rules, indexes, Auth providers, Netlify settings, DNS and external processors are outside repo	Static audit cannot confirm deployed controls/behavior	absent configuration	Manually verify production configuration using checklist below

Authentication

AUD-011 — Signup can strand an Auth identity

Issue. `createUserWithEmailAndPassword` succeeds before the user document, referral, profile stub, verification email and sign-out. Any later exception is displayed as a signup failure, but there is no resume/rollback.

Evidence. `web/src/app/signup/page.tsx:28-97`.

Why it matters. Retrying produces `auth/email-already-in-use`; the user may be signed in but incomplete and not know how to recover. A verification-send failure also skips the normal sign-out path.

Severity. ● High.

Recommendation. Make onboarding idempotent by UID, detect/resume incomplete accounts, and separate Auth creation success from profile provisioning status.

AUD-016 — Second-role write can lose the legacy role

Issue. Code derives `roles = [data.role]`, then calls `arrayUnion(newRole)` on a missing `roles` field. Firestore creates `[newRole]`, not `[oldRole, newRole]`; only the in-memory array contains both.

Evidence. `web/src/app/login/page.tsx:63-76`; equivalent Google signup logic at `signup/page.tsx:114-123`.

Why it matters. After refresh, the original dashboard rejects the user even though role-switching UI previously showed both.

Severity. ● Medium.

Recommendation. Write a canonical full roles array in a trusted migration/role service.

AUD-017 — Student/parent vocabulary breaks profile initialization

Issue. Public routes use `student`; signup branches create a parent profile only for `parent`.

Evidence. `web/src/app/signup/page.tsx:25,72-79,125-131,168-174`.

Why it matters. New consumer accounts follow different initialization depending on login/signup method and rely on later forms to repair missing documents.

Severity. ● Medium.

Recommendation. Choose one role name and one initialization service; migrate aliases explicitly.

AUD-007 — Profile document identity is inconsistent

Issue. Some Google login paths create `parents/{generatedId}` or `tutors/{generatedId}`, while signup/forms generally use `{uid}`. Fetchers sometimes query `authUid`, while handlers/listeners/deletion often use UID directly.

Evidence. `web/src/app/login/page.tsx:266-278`; `signup/page.tsx:72-79`; `dashboardApi.ts:267-272`; teacher handlers at `page.tsx:315,475,584,2958`.

Why it matters. A profile may load through `authUid` but receive no updates/listener events or survive deletion. Teacher-created applications can use UID while the fetcher queries the generated tutor document ID.

Severity. ● High.

Recommendation. Use Auth UID as canonical role-profile document ID or maintain a single authoritative mapping resolved everywhere.

Authorization

AUD-002 — Client-controlled role grants

Issue. Login appends the query-selected role. Dashboard fetchers create a missing user document with the dashboard's role. Visiting a role path is therefore part of role provisioning.

Evidence. `web/src/app/login/page.tsx:60-75,250-281`; `web/src/api/dashboardApi.ts:16-34,245-264`.

Why it matters. If Firestore Rules permit the same writes required by normal clients, an authenticated user can self-grant roles. UI selection is not an authorization ceremony.

Severity. ● High.

Recommendation. Move grants to an authenticated trusted service with explicit eligibility/approval and immutable audit history.

AUD-003 — Database authorization boundary is absent from source

Issue. No `firestore.rules`, Storage rules, Firebase config, or rule tests are committed, yet all important reads/writes use the Web SDK.

Evidence. Repository-wide file inventory; `web/src/utils/firebase/client.ts:1-18`.

Why it matters. Client route guards and button checks can be bypassed with direct SDK calls. The audit cannot determine whether production rules prevent IDOR, role escalation, wallet edits or cross-user reads.

Severity. ● High architectural/security gap; this is not a claim that deployed rules are definitely open.

Recommendation. Version rules and emulator tests with the app. Restrict financial/role/status fields to trusted service identities.

Privacy/IDOR exposure pattern

Teacher aggregation reads the entire `students` collection (`dashboardApi.ts:314-319`) and student aggregation reads all profiled tutors (`:56-59`). Stored records include contact/location data. Secure rules must make these queries legal without exposing unrelated private fields, which is difficult because rules do not redact fields. Split sanitized marketplace lead documents from private profiles.

Firestore and database integrity

AUD-004 — Group save contract mismatch can erase assignments

Issue. `GroupManager` reads and returns `student.groupId`. The dashboard passes raw students whose canonical field is `groupDocId`, compares `student.groupDocId`, and persists that undefined value.

Evidence. `web/src/components/GroupManager.tsx:39-49,188-200`; `web/src/app/dashboard/student/page.tsx:1991-2009,200-207`.

Why it matters. Existing students appear unassigned. Saving can clear `groupDocId` for every student, compute no new groups, and continue into deletion logic.

Severity. ● High.

Recommendation. Define a typed group-edit DTO and integration-test round trips before enabling persistence.

AUD-005 — Synchronizer deletes current-format groups

Issue. It reads `groupData.studentIds`; current groups write `studentDocIds`. Missing legacy data becomes an empty array and executes group deletion. It also queries students and requests by `groupId`, while current records use `groupDocId`.

Evidence. `web/src/utils/groupUtils.ts:12-24,48-60`; `DemoForm.tsx:603,641-676`.

Why it matters. Regrouping or learner removal calls this helper and can delete a valid group while failing to find/delete its current request.

Severity. ● High.

Recommendation. Never use field absence as proof of emptiness. Migrate aliases and transact validated membership changes.

AUD-006 — Onboarding writes unrelated group identities

Issue. Student records receive a temporary/grouping UI ID. Later each group is stored under a new Firestore auto-ID, and requests reference that auto-ID; students are never updated to it.

Evidence. `web/src/components/DemoForm.tsx:577-618,638-676`.

Why it matters. Queries and joins cannot connect students to their persisted group/request. The dashboard reconstructs synthetic groups that disagree with stored groups.

Severity. ● High.

Recommendation. Allocate `groupRef` before learner payloads and use `groupRef.id` for every relationship in one batch/transaction.

AUD-008 — Application workflows are partially commit-able

Issue. Application creation writes the application, tutor queue and each learner queue sequentially. Decline/payment/hire repeats this pattern in reverse.

Evidence. Student creation `page.tsx:704-737`; teacher creation `page.tsx:449-484`; student decline `:940-951`; teacher decline `:646-657`.

Why it matters. Network/rule failures leave queue counters and actual application status disagreeing. Queue cleanup itself is used to enforce business limits.

Severity. ● High.

Recommendation. Use an idempotent server command with Firestore transaction/batch and recoverable outbox work.

AUD-013 — Hiring cannot find competing current-format applications

Issue. Cleanup queries `applications.groupId` and `.studentId`; creation writes `groupDocId` and `studentDocIds`.

Evidence. `web/src/app/dashboard/student/page.tsx:980-1014,1057-1091` versus `:711-712,842-843`.

Why it matters. Multiple tutors may remain in active states and retain contact/access after another tutor is hired.

Severity. ● High.

Recommendation. Store a canonical engagement key and atomically close all competing applications by trusted indexed query.

AUD-014 — Learner deletion leaves application references

Issue. Removal first deletes the learner, then queries only `studentId`; current group applications use `studentDocIds`. Tutor/student pending arrays are not repaired.

Evidence. `web/src/app/dashboard/student/page.tsx:3458-3481`.

Why it matters. Applications continue referencing deleted personal data and count toward queues. If later cleanup fails, the learner is already gone.

Severity. ● High.

Recommendation. Resolve references first, require safe lifecycle state, then delete through an idempotent orchestrated workflow.

AUD-018/AUD-019 — Onboarding and edit paths use incompatible shapes

Silent replay writes one request per learner with `studentDocId`, no group record, and `category: formData.category`; the multi-step form stores category on each learner. The ordinary dashboard form instead creates group aggregates. Profile loading then searches legacy `parentId`, `studentId` and `groupId` fields.

Evidence. `web/src/app/dashboard/student/page.tsx:497-556`; `web/src/components/DemoForm.tsx:143-165,171,526,550,574-696`.

Impact. Data produced by one supported entry path cannot reliably be edited or discovered by another. Preferences appear empty and duplicate request documents accumulate.

Severity. ● Medium.

Recommendation. One schema-validating onboarding command must serve fresh form, replay, add learner and edit paths.

AUD-034 — Group ID is written into a student field

Teacher offer creation receives a grouped lead, then sets both `studentDocId` and `groupDocId` to `student.id` (the group ID), while separately storing real learner IDs in `studentDocIds`.

Evidence. `web/src/app/dashboard/teacher/page.tsx:456-459,564-567`; primary enrichment at `dashboardApi.ts:442-505`.

Impact. `studentDetails` and its skills can be missing, so UI and matching depend on inconsistent `studentsList` fallbacks.

Severity. ● Medium.

Recommendation. Make `groupDocId` required for group engagements, keep singular learner ID optional/accurate, and type the union.

AUD-043 — Referral attribution is not uniquely enforced

Referral codes use four name characters plus a UID substring; the signup query takes the first matching user. A nonmatching code is still stored in `referredBy` without feedback.

Evidence. `web/src/utlis/referral.ts:1-17`; `web/src/app/signup/page.tsx:47-69,143-165`.

Impact. Collision or invalid input produces incorrect/unresolvable attribution. Concurrent code assignment is not reserved.

Severity. ● Low.

Recommendation. Store normalized code documents with unique IDs and validate before committing referral metadata.

Payments and financial state

AUD-001 — The payment system has no trusted payment

Issue. Tutor “payment” directly writes `demoPaymentPaid` and advances status. Student “payment” optionally subtracts wallet and writes `feePaid`. Tutor can self-report monthly receipts. No provider SDK, order, charge, server verification, callback, signature, webhook or transaction ledger exists.

Evidence. `web/src/app/dashboard/teacher/page.tsx:302-369` ; `web/src/app/dashboard/student/page.tsx:1024-1102` ; repository/package inventory.

Why it matters. Any user able to execute permitted client writes can claim payment; retries and direct SDK calls alter financial state. UI displays success and unlocks contact/business transitions without evidence of money movement.

Severity. ● Critical if these screens are enabled as production payment functionality.

Recommendation. Disable financial claims until a server-calculated, provider-verified, idempotent payment and ledger system exists.

AUD-009 — Withdrawal has duplicate/lost-balance races

Both dashboards validate cached balance, create an auto-ID pending withdrawal, then set balance to zero. Two tabs can read the same balance and create two requests. Failure after creation leaves spendable balance; failure after an external payout but before state reconciliation is not representable.

Evidence. Student `page.tsx:1111-1127` ; teacher `page.tsx:677-693` .

Severity. ● High.

Recommendation. Use a server transaction with a unique active-withdrawal constraint and ledger reservation; validate destination and process asynchronously.

AUD-028 — Wallet deduction commits before payment state

Issue. Student payment subtracts wallet first, then updates the application and performs cleanup.

Evidence. `web/src/app/dashboard/student/page.tsx:1030-1091` .

Why it matters. A later permission/network failure charges wallet credit but reports payment failure. No transaction record shows the deducted amount.

Severity. ● Medium (subsumed by the Critical payment architecture but independently actionable).

Recommendation. Ledger debit and verified payment/application transition must be one idempotent server operation.

AUD-029/AUD-030/AUD-031 — Financial projections disagree with writes

The received-payment handler writes `subsequentPayments` ; history UIs read `paymentHistory` . Upcoming-class projections omit `feePaid` and `startDate` , but overdue code reads those top-level fields. Earnings count every `tuition_started` final price even when `feePaid` is false, estimate ₹100 demo fees, and use approximate dates.

Evidence. `teacher/page.tsx:351-363,2460-2675` ; `student/page.tsx:158-166,2561-2565,3288-3318` ; `dashboardApi.ts:218-231,522-588` .

Impact. Users see empty history, missing dues, and revenue that is not accounting-grade.

Severity. ● Medium.

Recommendation. Replace application-derived finance with one verified ledger schema and typed projections.

Business logic and concurrency

AUD-015 — Application state machine has no transition guard

Handlers accept an application ID and action, construct partial fields and call `updateDoc`. They do not read/check the current server state or revision. Client role/status rendering is the only sequencing mechanism visible in repo.

Evidence. Student `page.tsx:881-958`; teacher `page.tsx:603-664`.

Impact. Two participants can concurrently accept, decline, counter or schedule; last write wins and can create combinations such as decline after payment or stale counter after acceptance.

Severity.  High.

Recommendation. Centralize an allowlisted state machine with transaction precondition on current status/version and actor role.

AUD-023/AUD-024/AUD-025 — Expiry is client-clock write logic inside reads

Each dashboard fetch maps applications and may write declines, queue removals, or `waiting_for_parent_decision`. It names the parsed demo start as `demoEndTime`, uses start plus 48 hours, and silently ignores some failures. Both role fetchers duplicate the logic.

Evidence. `web/src/api/dashboardApi.ts:112-165,444-498`.

Impact. Nothing expires without dashboard traffic; incorrect device time can change state; simultaneous role loads repeat writes; invalid date/time strings produce inconsistent results; ordinary reads require write authorization.

Severity.  Medium.

Recommendation. Store authoritative start/end timestamps and run one scheduled/idempotent server transition worker.

AUD-026 — Limits are raceable snapshots

Daily requests are counted from cached applications; pending counts come from cached arrays with a follow-up query; demo limits use local time and cached applications. None is reserved atomically.

Evidence. Student `page.tsx:606-668,748-818`; teacher `page.tsx:374-424,495-547`.

Impact. Parallel tabs/devices or two users targeting the same queue pass the same checks and exceed limits. Failed queue writes also permanently distort counts.

Severity.  Medium.

Recommendation. Enforce counters/capacity in a transaction or trusted command using current database state.

AUD-027 — UI loading flags are not duplicate protection

Creation handlers test state flags, but state updates do not synchronously lock another event in the same tick and no request ID is persisted. Hire confirmation calls the async action without awaiting it and closes immediately.

Evidence. Student `page.tsx:657,690-745,806,819-878,3525-3533`; analogous teacher offer code.

Impact. Rapid activation, browser retries, effect replay or ambiguous network completion can produce duplicate applications/transitions.

Severity.  Medium.

Recommendation. Generate a durable idempotency key before submission and make the server operation unique by actor/engagement/command.

AUD-032 — Negotiation limits change by entry point

Priced tutor offers validate 100%-140% of learner budget and store those bounds. Direct tutor demo requests set minimum to 60% of offer and maximum to 120%, with no equivalent initial validation.

Evidence. `web/src/app/dashboard/teacher/page.tsx:425-464,555-573`.

Impact. Users can obtain materially different commercial terms by choosing “demo” instead of “offer”; counter validation later trusts stored bounds.

Severity. ● Medium.

Recommendation. Product-own a single initiator-based policy and test every creation/counter path against it.

AUD-033 — Recommendation implementations disagree

Student fetch filtering uses `(board OR class) AND subject`; teacher school matching uses `board OR class OR subject`; `calculateSuitabilityScore` uses additive scoring and a separate class parser. Pages then add their own filtering/sorting.

Evidence. `web/src/api/dashboardApi.ts:67-96,347-418`; `web/src/utils/matching.ts:9-95`; dashboard page scoring blocks.

Impact. The same tutor/learner can be available, recommended, and scored differently. Business behavior is difficult to explain and changes drift across copies.

Severity. ● Medium.

Recommendation. One pure typed matcher should return eligibility, score and human-readable reasons with fixtures for all categories.

AUD-049 — Marketing and workflow disagree on demo cost

The public FAQ states that the demo is “completely free” with “No payment required,” but teacher workflow presents “Complete Payment” and advances only after marking demo fee paid.

Evidence. `web/src/components/FAQ.tsx:40-42`; `web/src/app/dashboard/teacher/page.tsx:302-340,3008-3088`.

Severity. ● Low technical severity, potentially larger legal/support impact.

Recommendation. Clarify who pays, when, refund behavior and wording across product/legal content.

React, async behavior and reliability

AUD-012 — Silent form replay can duplicate records

On mount, an async effect reads persistent JSON and creates new auto-ID students/requests. The key is removed only at the end. There is no cancellation, in-progress record or deterministic document ID.

Evidence. `web/src/app/dashboard/student/page.tsx:474-567`; teacher replay `:218-272` is similarly uncanceled, though its UID updates are more naturally idempotent.

Impact. Development Strict Mode effect replay, remount, refresh, or partial failure re-executes writes. The catch logs but leaves data queued for future retries with no user explanation.

Severity. 🟡 High for student data duplication.

Recommendation. Persist a submission UUID/status, use deterministic IDs/transaction, remove or mark consumed before rendering success, and support recovery.

AUD-020/AUD-021/AUD-022 — Realtime lifecycle is fragile

Teacher fetch resolves a tutor document ID but listener uses UID. Listener setup dynamically imports before assigning `unsubscribe`; cleanup may run first. Neither listener supplies an error callback.

Evidence. `dashboardApi.ts:267-272`; student `page.tsx:569-585`; teacher `page.tsx:274-290`.

Impact. Legacy profiles get no live updates, unmounted components can leak listeners, and permission/network failures silently stop refresh.

Severity. 🟡 Medium.

Recommendation. Resolve one identity, attach with cancellation guard, handle errors, and test mount/unmount/account-switch cases.

AUD-044 — Referral generation cannot retry after failure

`isGeneratingRef` is set true and never reset. Successful cache mutation makes the guard irrelevant; failure leaves the missing code and permanent in-progress state for that mount.

Evidence. Student `page.tsx:448-472`; teacher `page.tsx:187-210`.

Severity. 🟢 Low.

Recommendation. Reset in `finally`, avoid including the full changing `data` object in dependencies, and expose a retry control.

AUD-045 — Student dashboard lacks a general load-error path

Auth/role errors redirect, but other SWR failures have no explicit error return equivalent to the teacher page. Once `isLoading` is false and `data` absent, the large UI renders through optional fallbacks.

Evidence. Student `page.tsx:145-153,1149-1153`; teacher `page.tsx:667-668`.

Impact. Permission/index/network failures can appear as an empty/broken dashboard without recovery guidance.

Severity. 🟢 Low.

Recommendation. Add route-level error state, retry and diagnostics-safe support reference.

AUD-040 — Meeting modal preserves stale link/platform

The open-reset effect resets value/date/time only, not platform/link. URL validation calls `new URL` without restricting scheme or trusted domain. Stored delimiter data is later used as an anchor.

Evidence. `web/src/components/ActionModal.tsx:39-49,69-80`; dashboard links at student `page.tsx:2085`, teacher `:1653`.

Impact. Reopening for another application can reuse the previous private meeting link, and arbitrary URL schemes/hosts enter persisted records (browser/React mitigations may block some dangerous schemes, but no application policy exists).

Severity. ● Medium.

Recommendation. Reset all state, accept only HTTPS and optionally provider host allowlists, and store structured meeting fields.

AUD-042 — Geocoding requests are not lifecycle-safe

All three form implementations perform direct fetches without AbortController, fetch timeout, `res.ok`, response validation or component-mounted checks. Browser geolocation has a 10-second timeout, but subsequent fetch does not.

Evidence. `TeacherForm.tsx:138-170`; `DemoForm.tsx:250-286`; `GroupSettingsModal.tsx:43-76,166-176`.

Impact. Dependency hangs or unmounts can leave loading state/stale form updates; malformed/rate-limit HTML causes JSON errors.

Severity. ● Low.

Recommendation. One cancellable geocoding client with schema validation, explicit attribution/rate policy and manual fallback.

API, Firebase query and performance audit

There is no application-owned HTTP API. Findings in this section concern the browser-side module named `dashboardApi.ts` and direct Firestore/Nominatim calls.

AUD-036 — Teacher aggregation scales and exposes by whole database

The teacher fetcher reads every `tuition_started` application, every application in three locked statuses, and every student. It filters/group-joins in browser memory.

Evidence. `web/src/api/dashboardApi.ts:275-319`.

Why it matters. Firestore billable reads and payload size scale with total platform activity per tutor page load, not that tutor's accessible page. If rules allow the queries, sensitive learner records reach every tutor client.

Severity. ● Medium.

Recommendation. Materialize sanitized available-lead projections and scoped lock/engagement state, queried with cursor limits.

AUD-037 — Snapshot events amplify reads

Listeners ignore snapshot contents and call `mutate()`. Each revalidation rereads user/profile, applications, students/tutors, groups/locks, referrals and all pricing. An expiry write during fetch can itself produce another snapshot.

Evidence. Student `page.tsx:576-579`; teacher `page.tsx:281-284`; both `dashboardApi` functions.

Impact. Negotiation bursts generate repeated broad queries, UI recomputation and possible feedback traffic. SWR's 60-second dedupe can conversely delay intended immediate refresh semantics depending on timing.

Severity. ● Medium.

Recommendation. Subscribe to a narrow collection/view, merge snapshot deltas, and separately cache low-change pricing/profile data.

AUD-038 — No pagination or query limits

Tutor, student, application, referral, group and pricing reads generally fetch entire matching collections. Marketplace lists map every result; no cursors or virtualization are present.

Evidence. `web/src/api/dashboardApi.ts:41-59,267-319,421-438`; dashboard list render branches.

Impact. Page latency, memory, DOM size and Firestore cost grow without bound. The schema analyzer already sampled 30 students and 27 pricing entries; production growth will magnify this pattern.

Severity. ● Medium.

Recommendation. Define ordered indexed cursor pagination, bounded recommendation windows and virtualized long lists.

N+1 and sequential patterns

Referenced tutors/students are fetched in chunks, which is better than one read per document, but chunks are executed sequentially. Cross-document queue removals and deletions loop with individual awaited writes. These patterns magnify latency and partial failure. They are captured primarily by AUD-008 and AUD-038 rather than duplicated as separate findings.

AUD-046 — Oversized raw dashboard asset

`web/public/book.png` is approximately 9.68 MB and appears through raw `<img>` in both marketplace loading/empty branches (`student/page.tsx:1700,1716`; `teacher/page.tsx:1334,1350`).

Severity. ● Low.

Impact. First-time users can download a multi-megabyte decorative asset without Next image optimization; mobile bandwidth and load time suffer.

Recommendation. Produce dimensions-appropriate WebP/AVIF variants and use optimized/lazy image delivery.

Input, URL and browser-storage audit

AUD-039 — Unvalidated post-auth destination

`next` comes from `useSearchParams / URLSearchParams` and is passed to `router.push` without checking it starts with a safe local route.

Evidence. `web/src/app/login/page.tsx:83-85,284`; `signup/page.tsx:182-184`.

Impact. Depending on Next router handling, an attacker-crafted auth link can navigate to an external or unintended destination after trust-establishing login. At minimum it allows route confusion.

Severity. 🟡 Medium.

Recommendation. Parse against a fixed origin and allowlist expected same-origin path prefixes; reject protocol-relative/backslash tricks.

XSS/injection result

No user-derived `dangerouslySetInnerHTML`, `eval`, dynamic function construction or string-built database query was found. The two raw JSON-LD injections use static in-source objects (`app/layout.tsx:73-97`, `FAQ.tsx:49-67`), so they are not an XSS finding. React escapes ordinary stored names/messages. The remaining URL concern is AUD-040. Classic SQL injection is not applicable to structured Firestore calls.

AUD-054 — Logout clears the entire origin storage

Both role pages call `localStorage.clear()` and `sessionStorage.clear()`.

Evidence. Student `page.tsx:588-594`; teacher `page.tsx:293-299`.

Impact. It removes unfinished profile handoff, selected category, and any future same-origin module's state, not just session metadata.

Severity. 🟢 Low.

Recommendation. Enumerate and remove owned keys; Firebase sign-out remains the actual session termination.

Sensitive browser storage

Pre-auth forms serialize contact, address, learner and tutor profile information into local storage (`DemoForm.tsx:413-420`; `TeacherForm.tsx:186-196`). This persists beyond tabs and is accessible to any same-origin script/XSS. It is not assigned a separate severity because the more material replay risk is AUD-012, but data minimization and short-lived/session storage should be part of that remediation.

Code quality and maintainability

God components and duplicated domain logic

Student dashboard is about 3,768 lines; teacher dashboard about 3,189; DemoForm about 1,704; TeacherForm about 1,166. Pages combine presentation, Firestore repositories, authorization assumptions, state machine, payments, analytics, deletion, notifications and matching. Status arrays, queue verification, referral effects, payment/withdrawal handlers, expiry and scoring are duplicated.

This is an architectural risk rather than a standalone counted issue: it explains AUD-015, AUD-023, AUD-029, AUD-032 and AUD-033. With pervasive `any`, TypeScript cannot detect field drift such as `paymentHistory / subsequentPayments` or `groupId / groupDocId`.

AUD-047 — Dependency/import surface does not match implementation

A static import scan found no source imports for Emotion, MUI, most Radix packages, canvas-confetti, date-fns, carousel, OTP, React DnD, React Hook Form, Recharts and others. Axios is imported in login and both dashboards but not called. `groupIds` in `dashboardApi.ts:427` and several local `user / logo` values are unused candidates.

Impact. Install time, vulnerability/audit surface, peer conflicts and lint noise grow. Some unused packages may be future-intent, so removal requires build verification.

Severity. ● Low.

Recommendation. Confirm with compiler/bundle analyzer, then remove in small tested batches and enforce unused-import linting.

AUD-050 — Encoding defects are visible in source

Examples include `â,`¹ in `ActionModal ( :137 )`, `Â@` in login/footer-like text, and corrupted emoji/comment text. This likely reflects an encoding conversion, not runtime business logic.

Severity. ● Low.

Recommendation. Standardize UTF-8 through `editorconfig`/CI and add screenshot/smoke coverage for currency/legal labels.

Hard-coded and dead concepts

Magic values include five daily/pending requests, two demos/seven days, 60/100/120/140 percent bounds, ₹100 fallback, ₹4000 fallback, 18% tax, ₹1000 withdrawal, 48-hour decision and 7/14-day locks. Status filters also mention legacy/unwritten values (`reviewing`, `offer_sent`, `demo_booked`, `accepted`, `confirmed`, `locked`). Legacy collections `tutor_requests` and `direct_requests` appear only in deletion. These are not individually counted, but centralizing them is necessary to prevent further drift.

Configuration and deployment

AUD-041 — Firebase configuration fails late

All seven values are copied from `process.env`; no assertion distinguishes required values from optional measurement/storage/messaging values.

Evidence. `web/src/utils/firebase/client.ts:5-18`.

Impact. A misconfigured preview/production build reaches user code and fails with Firebase errors or points at an unintended project.

Severity. ● Low.

Recommendation. Validate at build/startup, display no values, and explicitly bind environments to expected project IDs.

AUD-048 — Sitemap is incomplete

It returns only home and login, contrary to the available signup/legal public routes. `lastModified: new Date()` changes every request/build regardless of content.

Evidence. `web/src/app/sitemap.ts:3-17`; route inventory.

Impact. Search metadata is incomplete and falsely signals freshness.

Severity. ● Low.

Recommendation. List only intended public canonical routes with meaningful modification dates.

AUD-051 — Analyzer credential and exit behavior

`db_analyzer/analyze.js` requires a specifically named service-account JSON in its directory. Missing file exits 1, but errors inside `analyzeDatabase` are caught and only logged; the promise is invoked without a final rejection handler/exit code.

Evidence. `db_analyzer/analyze.js:5-18,112-133`.

Impact. Operators are encouraged to place a powerful credential beside source, and automation may treat a failed analysis as success.

Severity. ● Low (credential exposure becomes Critical if the file is ever committed; none was found).

Recommendation. Use Application Default Credentials or an explicit external path, add ignore/permission guidance, and set nonzero exit on analysis failure.

AUD-053 — Runtime and CI are unspecified

No `engines`, package-manager pin, `.nvmrc`, `.node-version`, GitHub/other CI pipeline or test gate exists. Netlify forces legacy peer-dependency handling.

Evidence. `web/package.json`; `netlify.toml`; repository inventory.

Severity. ● Low.

Recommendation. Pin a supported Node/npm toolchain and require clean install, lint, build and tests in CI/deploy previews.

AUD-056 — Executable checks remain unverified

The initial repository had no `node_modules`. `npm install` was attempted for 120 seconds and timed out with no completed binaries; subsequent `npm run lint` and `npm run build` reported `eslint` and `next` not recognized.

Severity. ● Informational.

Interpretation. This does not prove the code passes or fails compilation/lint. Source evidence such as unused imports is reported as static evidence, not lint output.

Testing audit

AUD-052 — No tests cover the highest-risk behavior

There is no `test` script or application Jest/Vitest/Playwright/Cypress/Firebase Emulator configuration. Tests under `.gemini/skills` test tool assets, not the website.

Evidence. `web/package.json`; repository inventory.

Why it matters. Role rules, group round trips, alias compatibility, state transitions, payment/withdrawal idempotency, deletion and concurrent clients have no regression defense. Multiple confirmed mismatches are exactly the type a single integration test would expose.

Severity. 🟡 High because it surrounds security/financial/data-corruption paths, though absence of tests alone is not a runtime vulnerability.

Risk-prioritized recommendation. First add Firestore Emulator rule/ownership tests; then group/onboarding integration tests; application transition concurrency tests; payment/withdrawal server tests; two-browser-role E2E; and pure matching/ID/error unit tests. Use isolated emulator fixtures for UID and legacy generated-ID shapes.

Dependencies

Resolved core versions were documented from the lockfile, but vulnerability freshness is time-sensitive and a networked package audit did not complete. The static issue is AUD-047. Manual verification should run `npm ci`, `npm audit` (with human triage, not automatic upgrades), license review, and a production bundle analyzer. Package presence does not imply use: notably there is no payment SDK despite payment UI.

Reliability and recovery assessment

Flow	Failure after first successful write	Current recovery
Email signup	Auth exists, Firestore/profile may not	Retry conflicts; manual login/form repair
Student onboarding	Some learners/groups/requests exist	Stored replay reruns and can duplicate
Application creation	App exists but one or more queues missing	Toast error; no rollback/reconciler
Decline/expiry	Status changed but queues stale	Future queue verification may clean only sampled aliases
Hire/payment	Target changed but alternatives/notification fail	Toast may report failure after irreversible partial work
Withdrawal	Request exists but wallet still available, or balance zero after request only	No processor/reconciliation UI
Learner/account deletion	Primary records gone, related/Auth deletion fails	Error toast or false success; no restore
Nominatim	Browser location obtained but lookup fails/hangs	Manual address entry after alert in some paths

No offline mode, write queue UX, operation status page, reconciliation job, remote error monitoring, audit correlation ID or administrator recovery console is present in this repository.

Notifications

AUD-035 — Two notification models are disconnected

Dashboard “notifications” are application projections assembled in `dashboardApi.ts:180-188,514-518`. Hiring/payment cleanup also creates `notifications` documents at student dashboard lines `1000-1010` and `1077-1087`, but no website query reads that collection and no mark-read action exists.

Why it matters. Stored notifications consume writes without delivery, while derived notifications reappear indefinitely with no durable unread state. The stored `userId` uses tutor document identity, which is itself inconsistent under AUD-007.

Severity. ● Medium.

Recommendation. Emit one trusted notification event per state transition with recipient Auth UID, deduplication key, read timestamp and explicit in-app/channel consumers.

Notification creation failures are deliberately swallowed with `.catch(console.error)`, so the main state can succeed without notification. This is acceptable only if a retryable outbox exists; none does.

Account and data deletion

AUD-010 — Account deletion is irreversible before it is authorized to finish

Both role pages delete profile, user and related application records sequentially, then call `deleteUser`. Firebase commonly requires recent authentication for that final step. The handler treats that specific failure as successful account deletion, signs out and redirects.

Evidence. Student `page.tsx:3577-3631`; teacher `page.tsx:2948-2992`.

Why it matters. The identity remains usable after reauthentication but its base data is gone. Earlier query/write failure creates a different partial deletion. Related groups, notifications, referrals, withdrawals and pending arrays are not comprehensively cleaned.

Severity. ● High.

Recommendation. Require reauthentication first, create an auditable deletion job, disable access, delete by canonical reference inventory, support retries, then delete Auth last and report actual completion.

Compound risks

Client trust chain: role grant + direct Firestore + absent rules

```
Query-selected/self-created role
+ dashboard-only authorization checks
+ direct document IDs in browser handlers
+ production rules unavailable for review
= possible cross-role/IDOR exposure if deployed rules are permissive
```

No individual unknown proves exploitation. Together they mean the repository itself supplies no trustworthy authorization story. A penetration test must attempt direct SDK reads/writes, not merely navigate the UI.

Financial chain: fake success + direct balance + no idempotency

No payment provider verification

- + client-writable payment flags/wallet
- + withdrawal auto-ID documents
- + non-transactional ordering
- + no ledger/reconciler
- = financial loss, duplicate payout, and irreconcilable records

This combination drives AUD-001 to Critical. Adding only a checkout widget would not solve wallet, webhook, idempotency or reconciliation.

Group corruption chain: three independent aliases

GroupManager returns groupId

- + dashboard persists groupDocId
- + normal onboarding uses temporary IDs then group auto-IDs
- + synchronizer reads studentIds/groupId
- + empty alias triggers delete
- = cleared learner links, deleted groups, orphan requests

These are not cosmetic naming inconsistencies. The normal save/remove execution paths invoke the incompatible helper.

Stale marketplace chain

Application write partially succeeds

- + pending arrays are counters
- + cleanup queries legacy fields
- + notification collection is not consumed
- + snapshot refetches broad stale projections
- = users see active/locked/declined states that disagree across roles

Account-erasure chain

Client-side multi-collection deletes

- + canonical ID drift
- + related collections omitted
- + Auth recent-login check occurs last
- + failure reported as success
- = both incomplete erasure and orphan login identity

Architectural risks

Current architectural problems

1. **Untrusted client is the domain service.** The browser calculates money, authorization-adjacent roles, limits and lifecycle transitions. This is already a security/reliability problem, not merely future scaling concern.
2. **No canonical domain schema.** any objects and alias-rich Firestore documents are the integration contract. Confirmed group, role and payment-history bugs result directly.
3. **Read model mutates write model.** Fetching dashboards can expire applications, coupling permissions, traffic and time automation.
4. **Denormalization lacks ownership/reconciliation.** Student/group/request/application/pending-array copies have no transaction or repair job.
5. **God pages combine every layer.** A single UI edit can affect database, security, finance and both roles; behavior cannot be unit-tested in isolation.
6. **Whole-database marketplace reads.** Teacher views already depend on global student/application reads.

Growth risks

- Firestore vendor dependence is not inherently a defect, but direct Web SDK schemas permeate components, making a backend or database migration expensive.
- Status strings and business constants duplicated across two large pages will continue to drift.
- Contact data mixed into marketplace records makes privacy restriction harder as tutor count grows.
- Realtime full refetch and unbounded lists will produce nonlinear cost/user experience under higher activity.
- Without immutable event/ledger records, support cannot reliably reconstruct disputes.

Recommended target boundaries

Use Firebase Auth for identity; a trusted API/Functions layer for role grants, application commands, money, notifications, expiry and deletion; Firestore Rules as defense in depth; typed repositories/read models for profiles and paginated public leads; transactional command IDs/versioned state machine; immutable payment/wallet ledger; and scheduled/outbox workers for expiry/notifications. UI components should consume typed commands/results rather than import Firestore mutators.

Second-pass findings and validation

The second pass intentionally used different questions and evidence searches:

- Compared every group field emitted by UI, written by form/dashboard, and queried by helper. This surfaced AUD-004 through AUD-006 as three separate defects rather than one naming smell.
- Compared write fields against read/display fields. This found subsequentPayments versus paymentHistory (AUD-029) and the projected-class dues bug (AUD-030).
- Compared resolved profile IDs against handler/listener/delete IDs. This strengthened AUD-007 and found AUD-020.
- Compared each application creation path's bounds and participant fields. This found AUD-032 and AUD-034.
- Inspected async effect cleanup rather than only synchronous handlers. This found replay duplication and listener attachment race (AUD-012/AUD-021).
- Scanned raw assets and import/package usage. This found AUD-046/AUD-047.
- Searched dynamic HTML, URL construction and redirect inputs. Static JSON-LD was cleared as a false positive; next and meeting URLs remained AUD-039/AUD-040.
- Compared public claims with workflow state. This found AUD-049.

- Checked all special config/secret-like filenames. No secret file, rules, middleware or error boundary was present.

Potential false positives removed or bounded:

- Firebase Web API configuration names are not treated as secrets; no values are documented.
- Client-side route guards are not called an authentication bypass by themselves; actual data access depends on unknown deployed rules.
- Static `dangerouslySetInnerHTML` JSON-LD is not reported as XSS because the objects contain no user input.
- Missing Firestore indexes are an unknown, not a confirmed outage; most visible queries use single filters.
- Dependency vulnerabilities are not asserted without a completed networked audit.
- Nominatim failure remains Low because manual entry exists and geolocation has its own timeout.

Areas requiring manual verification

Firebase production controls

- Obtain deployed Firestore Rules and test them against attacker, unrelated student, participant tutor, unrelated tutor and admin/service identities.
- Verify whether users can update `roles`, wallet/payment fields, application participant IDs/status, other users' profiles, notifications and withdrawals directly.
- Verify query/rule compatibility for all-student and all-profiled-tutor reads.
- Obtain index definitions and production query error logs.
- Confirm App Check status, Auth authorized domains/providers, password policy and Google OAuth configuration.

Runtime and data

- Run a Firestore export/anonymized analysis to quantify UID/generated profile IDs and every alias (`groupId` / `groupDocId`, etc.).
- Reproduce group create, regroup, learner removal and request synchronization against an isolated emulator before touching production.
- Test two clients concurrently countering, accepting, paying and withdrawing.
- Verify actual timezone behavior for stored demo date/time formats and malformed legacy records.
- Determine whether Android/admin clients own fields/collections and what compatibility they require.

Payments and business operations

- Confirm whether payment UI is enabled in production and whether an external process not represented here verifies records.
- Confirm provider accounts, webhook endpoints, reconciliation, tax treatment, refunds, wallet funding, withdrawal approval and payout controls.
- Reconcile "free demo" policy with tutor fee behavior and legal PDFs.
- Determine subscription entitlement source; UI offers "Payments coming soon," while queue logic trusts `isSubscribed` from profile data.

Deployment/operations

- Run a clean locked install, lint, production build and bundle analysis in CI.
- Verify Netlify adapter/settings, Node version, environment-to-Firebase-project mapping, preview isolation, DNS and rollback procedure.
- Inspect runtime logs for permission/index errors, duplicate records, listener failures and payment/deletion support incidents.
- Confirm backup, retention, deletion and incident-response procedures.

Informational architecture facts

AUD-055 — No trusted application backend is checked in

No App Router API routes, server actions, Cloud Functions, Admin SDK web service, Storage usage, Messaging/service worker or payment provider integration exists. `db_analyzer` is an offline administrative utility only.

Severity.  Informational, but it sets the interpretation of every client-side control.

AUD-057 — Schema report is evidence, not schema

The analyzer samples at most 100 root documents per collection, ignores subcollections and only inspects the first object in an array. It was generated at one point in time.

Evidence. `db_analyzer/analyze.js:67-108`; `schema_report.md` header.

Severity.  Informational.

AUD-058 — Important production state is external

Firestore Rules/indexes, Firebase console settings, Netlify project configuration, DNS, provider dashboards and runtime data cannot be concluded from static source.

Severity.  Informational. Absence from source is not evidence of safety or insecurity; it requires the manual verification above.

Remediation priorities

Immediate containment

1. Treat current payment/wallet/withdrawal UI as non-production until verified server processing exists.
2. Disable or guard group editing/removal paths until canonical field fixes are migrated and tested.
3. Audit deployed Rules immediately; prohibit client role, wallet, payment and unrestricted status writes.
4. Require recent authentication before any account data deletion.

Next engineering milestone

1. Define canonical typed identities and migrate aliases.
2. Introduce trusted idempotent commands for applications, grouping, financial operations and deletion.
3. Add emulator Rules/integration tests before refactoring UI.
4. Separate sanitized marketplace leads from private contact profiles.

5. Add immutable ledger/outbox/audit records and operational reconciliation.

Stabilization and scale

1. Decompose dashboards by feature and centralize status/matching/constants.
2. Add pagination, scoped queries, narrow cache invalidation and optimized assets.
3. Add monitoring, structured errors, recovery jobs and CI quality gates.
4. Reconcile product/legal copy with implemented workflows.

Final audit statement

The repository implements a substantial two-sided marketplace UI, but its current trust and consistency boundaries are not production-safe for financial or adversarial use based on code alone. The most serious concerns are demonstrable client-side payment simulation, destructive group-field mismatches, non-atomic lifecycle operations, inconsistent identity models, and an unverifiable external Firebase authorization boundary. No application code was changed during this audit.